



HopeDrop – Blood Bank Management System

DEVELOPER GUIDE

Version: 1.0

Date: September 2026

Team: Group 50 – DevDynamos

Module: CO2060

Stack: Django 5.2 + React 19 + Vite 7

GitHub: github.com/cepdnaclk/e22_co2060_bloodmsystem

Table of Contents

1. Project Overview

2. Repository Structure

3. Development Environment Setup

- 3.1. Prerequisites
- 3.2. Backend Setup (Django)
- 3.3. Frontend Setup (React/Vite)
- 3.4. Key Configuration Settings
- 3.5. Running Both Servers

4. Architecture Overview

- 4.1. System Architecture Layers
- 4.2. Authentication Flow (JWT)
- 4.3. API URL Structure

5. Backend Deep Dive

- 5.1. Django App Structure Pattern
- 5.2. Custom User Model and Roles
- 5.3. Data Models Reference
- 5.4. API Endpoints Reference
- 5.5. 4-Stage Inventory Approval Workflow
- 5.6. Blood Lifecycle State Machines
- 5.7. Management Commands
- 5.8. Django Admin (Jazzmin)

6. Frontend Deep Dive

- 6.1. React App Structure
- 6.2. Routing and Role Guards
- 6.3. Authentication Context
- 6.4. API Service Layer (Axios)
- 6.5. Key Frontend Libraries

7. Database Design

8. Testing

- 8.1. Running Django Tests
- 8.2. Code Quality and Linting
- 8.3. CI/CD Pipeline (GitHub Actions)

9. Deployment

- 9.1. Docker
- 9.2. Production Checklist

10. Adding New Features – Developer Workflow

11. Troubleshooting for Developers

1. Project Overview

HopeDrop is a full-stack Blood Bank Management System built for Sri Lanka. It provides a digitised end-to-end workflow for managing blood donations, inventory, expiry, and hospital requests across a national network of hospitals and blood camps.

Key architectural decisions:

- Monorepo: /frontend (React) and /backEnd (Django) in one repository
- RESTful API only: Django serves JSON; React consumes it via Axios
- JWT authentication with rotating refresh tokens and blacklisting on logout
- Role-based access control enforced in API (permissions) and frontend (RoleRoute guard)
- SQLite for development; PostgreSQL-compatible via psycopg2 for production
- Docker support via Dockerfiles in /frontend and /backEnd/main
- GitHub Actions CI/CD pipeline for automated testing and linting

2. Repository Structure

```
e22-co2060-Blood-Bank-management-System-web-app/
  backEnd/
    main/                                # Django project root (manage.py is here)
    config/                              # settings.py, urls.py, wsgi.py, asgi.py,
  deployment.py
    api/v1/urls.py                        # API route aggregator for all apps
    apps/
      UserAuth/                          # Custom User model, auth endpoints, Hospital
  model
    models/                              # models.py, hospital.py, location.py
    services/                            # Views: register, login, profile, logout
    serializer/                          # DRF serializers
    urls.py                              # Auth URL patterns
    blood/bloodinventor/                 # Blood inventory models and APIs
      models/                            # bloodinventor.py, bloodBank.py,
  bloodRequest.py,
                                     # expiryAlert.py, inventoryChangeRequest.py
    services/                            # hospital/, officer/, admin/, public/ service
  files
    urls.py, urls_admin.py, urls_officer.py, urls_public.py
    management/commands/check_blood_expiry.py # daily expiry checker
    donor/                               # Donor, camp, collection models and APIs
      models/                            # donorDetails.py, bloodCamp.py,
  campRegistration.py,
                                     # campBloodCollection.py, donationHistory.py,
                                     # donorAlert.py, workflowNotification.py
    services/                            # bloodCampService.py, donorService.py, etc.
    urls/urls.py                         # Donor URL patterns
    medicalOfficers/                    # Doctor and StaffProfile models
      models/                            # doctor.py, hospitalStaff.py, doctor_message.py
    adminDashboard/                     # Admin-specific aggregated views
    manage.py
    requirements.txt                     # Python dependencies
  frontend/
    src/
      pages/                             # Page components by role:
      admin/                             # AdminDashboard, NationalInventory,
  DoctorsList,
                                     # HospitalManagement, ExpiryAlerts,
  BloodRequests...
    donor/                               # DonorDashboard, DonorEligibility,
  DonorNotifications...
    doctor/                             # DoctorDashboard
    bloodcamp/                          # CampDashboard, BloodCamps
    staff/                              # StaffDashboard, LabDashboard
    patient/                            # PatientDashboard
```

```
    public/                # LandingPage, AboutUs, Services, ContactPage,
Events...
    auth/                  # Login, SignUp, ForgotPassword, ResetPassword
    components/            # Reusable UI components (layout, nav, utils)
    api/                   # Axios config, RoleRoute.jsx
    context/               # AuthContext.jsx (global JWT state)
    hooks/                 # Custom React hooks
    services/              # Axios API call functions grouped by feature
    layouts/               # AdminLayout.jsx, MainLayout.jsx
    index.html, vite.config.js, package.json, tailwind.config.js,
eslint.config.js
    docs/                  # Jekyll GitHub Pages documentation site
    meeting notes/         # Weekly meeting PDFs (m1-m9)
    .flake8                # Python flake8 linting config
    .github/               # GitHub Actions CI/CD workflows
```

3. Development Environment Setup

3.1 Prerequisites

Tool	Version	Purpose
Python	3.11+	Backend runtime
Node.js	18+	Frontend runtime
npm	9+	Frontend package manager
Git	Any	Version control
pip	Latest	Python package manager
SQLite	Built-in with Python	Development database
Docker	Optional	Containerised deployment

3.2 Backend Setup (Django)

Navigate to the Django project directory:

```
cd backEnd/main
```

Create and activate a virtual environment:

```
python -m venv venv
```

```
# Windows PowerShell:
```

```
.\venv\Scripts\Activate.ps1
```

```
# Windows CMD:
```

```
.\venv\Scripts\activate.bat
```

```
# Linux/macOS:
```

```
source venv/bin/activate
```

Install all Python dependencies:

```
pip install -r ../requirements.txt
```

Apply database migrations:

```
python manage.py migrate
```

Create superuser (admin account):

```
python manage.py createsuperuser
```

Start the development server:

```
python manage.py runserver
```

Backend available at: <http://127.0.0.1:8000/>

Admin panel (Jazzmin) at: <http://127.0.0.1:8000/admin/>

TIP: Use the Makefile for common tasks: 'make migrate', 'make test', 'make lint' (check backEnd/main/Makefile).

3.3 Frontend Setup (React/Vite)

In a new terminal window:

```
cd frontend
```

```
npm install
```

```
npm run dev
```

Frontend available at: <http://localhost:5173/>

npm Command	Description
npm run dev	Start Vite dev server with hot module replacement (HMR)
npm run build	Build production-optimised bundle to dist/
npm run preview	Preview the production build locally
npm run lint	Run ESLint across all .jsx and .js source files

3.4 Key Configuration Settings

Backend key settings in backEnd/main/config/settings.py:

Setting	Default	Notes
DEBUG	True	Must be False in production
SECRET_KEY	Insecure dev key	Use a 50-char random key in production
ALLOWED_HOSTS	[]	Add domain/IP in production
CORS_ALLOWED_ORIGINS	localhost:5173,5174,3000	Update for production domain
CORS_ALLOW_ALL_ORIGINS	True	Remove or set False in production
DATABASE_ENGINE	sqlite3	Change to postgresql for production
ACCESS_TOKEN_LIFETIME	15 minutes	JWT access token duration
REFRESH_TOKEN_LIFETIME	1 day	JWT refresh token duration
ROTATE_REFRESH_TOKENS	True	Issues new refresh token on every use
BLACKLIST_AFTER_ROTATION	True	Blacklists old refresh token after rotation
EMAIL_BACKEND	console.EmailBackend	Change to smtp.EmailBackend for real emails

WARNING: NEVER commit SECRET_KEY, EMAIL_HOST_PASSWORD, or database credentials to Git. Use .env files with python-dotenv.

3.5 Running Both Servers Simultaneously

```
# Terminal 1 - Django Backend
cd backEnd/main
python manage.py runserver
```

```
# Terminal 2 - React/Vite Frontend
cd frontend
npm run dev
```

CORS is already configured to allow requests from localhost:5173. The frontend calls the API at <http://127.0.0.1:8000/api/v1/> by default.

4. Architecture Overview

4.1 System Architecture Layers

Layer	Technology	Role
Presentation	React 19 + Vite 7	SPA served by Vite; Axios calls backend REST API
API Gateway	Django REST Framework	Exposes all data as JSON REST endpoints at /api/v1/
Business Logic	Django Views/Services	Encapsulated in service files per app
Data Access	Django ORM	SQLite (dev) or PostgreSQL (prod) via Django models
Authentication	SimpleJWT	JWT tokens; refresh rotated and blacklisted on logout
Static Files	WhiteNoise	Serves collected static files in production

4.2 Authentication Flow (JWT)

- User POST /api/v1/auth/token/ with email + password
- Backend validates credentials, returns access_token (15 min) and refresh_token (1 day)
- Frontend stores tokens in localStorage via AuthContext
- Every authenticated API request includes: Authorization: Bearer <access_token>
- When access token expires, frontend POSTs /api/v1/auth/token/refresh/ with the refresh token
- On logout, POST /api/v1/auth/logout/ blacklists the refresh token (prevents reuse)

TIP: Token rotation is enabled: every refresh call issues a NEW refresh token and blacklists the old one, preventing replay attacks.

4.3 API URL Structure

All API endpoints are prefixed with /api/v1/ (configured in config/urls.py):

URL Prefix	Django App	Description
/api/v1/auth/	apps.UserAuth	Registration, login, logout, profile, hospital management
/api/v1/blood/	apps.blood.bloodinventor	Blood inventory CRUD, expiry alerts, change requests, national dashboard
/api/v1/medicalOfficers/	apps.medicalOfficers	Doctor profile photo upload
/api/v1/adminDashboard/	apps.adminDashboard	Admin-specific aggregated statistics views
/api/v1/donor/	apps.donor	Donor profile, blood camps, registrations, notifications, collections
/admin/	Django Admin	Jazzmin admin panel (superuser only)

5. Backend Deep Dive

5.1 Django App Structure Pattern

Each Django app follows the service-oriented pattern:

File/Folder	Purpose
models/	Django ORM models, one file per model for clarity and maintainability
services/	Business logic encapsulated in view functions or APIView subclasses
serializers/	DRF serializers for request validation and response transformation
urls.py	URL patterns mapping endpoints to view functions for this app
admin.py	Django admin site model registrations
tests.py	Unit and API integration tests
migrations/	Auto-generated Django migration files (do not edit manually)
permissions/	Custom DRF permission classes for role-based access control
selectors/	Read-only query functions separated from write business logic

5.2 Custom User Model and Roles

The system uses a custom User model (apps.UserAuth.models.User) extending AbstractUser:

```
class User(AbstractUser):
    DONOR = 'donor'
    DOCTOR = 'doctor'
    ADMIN = 'admin'
    INVENTORY_OFFICER = 'Inventor'
    BLOODCAMP = 'bloodcamp'

    role = models.CharField(max_length=10, choices=ROLE_CHOICES,
default=DONOR)
    email = models.EmailField(unique=True)
    username = models.CharField(max_length=10, unique=True)
    USERNAME_FIELD = 'email' # Users log in with email, not username
```

Role is used for:

- Post-login redirect to role-specific dashboard (AuthContext in frontend)
- API access restriction via custom DRF permission classes per view
- Filtering querysets (e.g., only bloodcamp role can create BloodCamp records)
- Limiting ForeignKey choices (e.g., camp organiser limit_choices_to={'role': 'bloodcamp'})

5.3 Data Models Reference

Model	App	Key Fields	FK / Related Models
User	UserAuth	username, email, role	Profile (1:1)
Profile	UserAuth	fullName, nic_number, blood_group, phoneNumber	User FK, Hospital FK, Country FK, District FK
Hospital	UserAuth	hosName, location, contact	BloodBank (1:1)
Country	UserAuth	name	Profile (FK)
District	UserAuth	name, country	Profile (FK)
DonorDetails	donor	qr_id (UUID), last_donation_date, is_eligible, total_donations, achievements	User (1:1)
BloodCamp	donor	title, date, start_time,	User organizer FK, Hospital

		end_time, location, status, total_collected_units	destination_hospital FK
CampRegistration	donor	status, appointment_time, arrived_at, screened_at, screened_by, rejection_reason	DonorDetails FK, BloodCamp FK
CampBloodCollection	donor	blood_type, units, status, transit timestamps, inventory_record	BloodCamp FK, CampRegistration (1:1), DonorDetails FK, Hospital FK
DonationHistory	donor	donation_date, hospital_name, blood_group, units, status, certificate_id (UUID)	DonorDetails FK, Hospital FK
DonorAlert	donor	message, alert_type, is_read, related_url	DonorDetails FK
WorkflowNotification	donor	event_type, message, is_read, metadata (JSON)	User FK
BloodBank	blood	name, hospital	Hospital (1:1)
BloodInventory	blood	blood_type, quantity, status, source_type, expiry_date, temperature, expiry_alert_sent	BloodBank FK, Hospital FK, BloodCamp source_camp FK
BloodRequest	blood	blood_group, units_requested, units_approved, priority_level, urgency, status, notes	Doctor FK, StaffProfile verified_by FK, StaffProfile approved_by FK
ExpiryAlert	blood	alert_type, blood_type, units_affected, days_until_expiry, is_resolved, resolved_action	BloodInventory FK, Hospital FK
InventoryChangeRequest	blood	action, status, blood_type, quantity_delta, reason, dates, 4 reviewer FKs	BloodInventory FK, BloodBank FK, User FKs (4 roles)
Doctor	medicalOfficers	full_name, email, specialization, license_number, is_active, credentials_created	User (1:1)
StaffProfile	medicalOfficers	employee_id, designation (TEC/MANG/NURSE), shift, branch_location	User (1:1)

5.4 API Endpoints Reference

Authentication Endpoints (/api/v1/auth/):

Method	Endpoint	Description	Auth
POST	/auth/register/	Create new user account with profile	No
POST	/auth/token/	Login – returns JWT access + refresh tokens	No
POST	/auth/token/refresh/	Refresh access token using refresh token	No
POST	/auth/logout/	Logout and blacklist refresh token	Yes
GET	/auth/profile/	Get current user's full profile	Yes
PUT	/auth/profile/update/	Update user profile fields	Yes
GET	/auth/getuser/	Get basic user info (id, email, role)	Yes

GET/POST	/auth/hospitals/	List all or create new hospital	Yes (Admin)
GET	/auth/hospitals/resolve/	Find hospital by name	Yes
POST	/auth/password-reset/	Request password reset email	No

Blood Inventory Endpoints (/api/v1/blood/):

Method	Endpoint	Description	Auth
GET	/blood/live-stock/	Get current national live blood stock (8 types)	No
GET	/blood/hospital/<id>/stock/	Get stock summary for a specific hospital	Yes
GET	/blood/hospitals/stock/	Get stock for all hospitals	Yes
GET	/blood/national/live/	National live stock summary	Yes
GET	/blood/national/dashboard/	Full national dashboard data and analytics	Admin
GET	/blood/hospital/<id>/inventory/	Detailed unit-level inventory for a hospital	Admin
GET	/blood/expiry/alerts/	List all expiry alerts (filterable)	Yes
POST	/blood/expiry/alerts/<id>/resolve/	Resolve an expiry alert with action	Yes
POST	/blood/officer/change-requests/	Officer: submit new inventory change request	Inventor
GET	/blood/officer/change-requests/my/	Officer: list own change requests	Inventor
GET	/blood/admin/change-requests/pending/	Admin: list all pending change requests	Admin
POST	/blood/admin/change-requests/<id>/adminDashboard-review/	Stage 1: admin initial review	Admin
POST	/blood/admin/change-requests/<id>/quality-check/	Stage 2: quality check	Admin
POST	/blood/admin/change-requests/<id>/final-approve/	Stage 3: final approval	Admin
POST	/blood/admin/change-requests/<id>/reject/	Reject at any stage with note	Admin
POST	/blood/admin/change-requests/<id>/execute/	Execute approved change request	Admin

Donor and Camp Endpoints (/api/v1/donor/):

Method	Endpoint	Description	Auth
GET/PUT	/donor/profile/	Donor profile read and update	Donor
GET	/donor/dashboard/	Donor dashboard stats and summary	Donor
GET	/donor/donations/	Donor's full donation history	Donor
GET	/donor/alerts/	List donor alerts	Donor
PATCH	/donor/alerts/<id>/read/	Mark specific alert as read	Donor
GET	/donor/notifications/	Workflow notifications list	Donor
PATCH	/donor/notifications/<id>/read/	Mark notification as read	Donor

GET	/donor/public/<qr-uuid>/	Public donor profile by QR ID	No
GET	/donor/camps/upcoming/	List upcoming blood camps	No
GET	/donor/camps/public/latest/	Latest public blood camp info	No
GET/POST	/donor/camps/	Organizer: list own or create camp	Bloodcamp
POST	/donor/camps/<id>/register/	Donor: register for a camp	Donor
GET	/donor/camps/<id>/registrations/	Organizer: list camp registrations	Bloodcamp
GET	/donor/camps/registrations/screening/	Organizer: screening queue	Bloodcamp
POST	/donor/camps/registrations/<id>/arrive/	Mark donor as arrived	Bloodcamp
POST	/donor/camps/registrations/<id>/screening/	Send donor to screening	Bloodcamp
POST	/donor/camps/registrations/<id>/approve/	Approve donor screening	Bloodcamp
POST	/donor/camps/registrations/<id>/reject/	Reject donor with reason	Bloodcamp
POST	/donor/camps/registrations/<id>/donate/	Record blood donation	Bloodcamp
GET	/donor/camps/donated-history/	Organizer: view donation history	Bloodcamp
GET	/donor/camps/<id>/collections/	List blood collections for camp	Yes
POST	/donor/camps/<id>/collections/dispatch/	Dispatch all blood to hospital	Bloodcamp
GET	/donor/camp-blood/pending/	List pending camp blood at hospital	Inventor
POST	/donor/camp-blood/<id>/receive/	Receive camp blood at hospital	Inventor
POST	/donor/camp-blood/<id>/verify/	Quality check camp blood	Inventor

5.5 4-Stage Inventory Approval Workflow

InventoryChangeRequest state machine:

Stage	Status Value	Triggered By	Actor
1	PENDING_ADMIN_REVIEW	Change request created by Inventor	System auto on creation
2	PENDING_QUALITY_CHECK	Admin passes initial review	Admin (admin-review endpoint)
3	PENDING_FINAL_ADMIN_APPROVAL	Quality check passed	Quality Reviewer (quality-check endpoint)
4	APPROVED_READY_TO_EXECUTE	Final admin sign-off given	Final Admin (final-approve endpoint)
5	EXECUTED	Approved change applied to BloodInventory	Admin (execute endpoint)
X	REJECTED	Any reviewer rejects at any stage	Reviewer – note required
X	CANCELLED	Inventor cancels before execution	Inventor

NOTE: Each stage has a dedicated API endpoint. All 4 reviewer FK fields are populated at each stage to maintain a complete audit trail.

5.6 Blood Lifecycle State Machines

CampBloodCollection – Full transit lifecycle:

```
collected
-> in_transit      (dispatch_camp_blood called by organizer)
-> received        (receive_camp_blood called by staff)
-> quality_check   (verify_camp_blood called - passes)
-> added_to_inventory (BloodInventory record auto-created)
-> rejected        (verify_camp_blood called - fails; rejection_reason
recorded)
```

BloodInventory – Status choices:

```
available (default when added)
-> reserved (allocated to a blood request)
-> used      (consumed/dispensed)
-> expired   (past expiry date)
-> damaged   (physically damaged)
-> deleted   (administratively removed)
```

BloodRequest – Approval lifecycle:

```
PENDING
-> VERIFIED      (lab staff confirms clinical need)
-> APPROVED      (admin allocates blood)
-> IN_PROGRESS   (blood being prepared)
-> FULFILLED     (blood delivered)
-> REJECTED      (denied at any point)
-> CANCELLED     (doctor cancels)
```

5.7 Management Commands

Custom daily expiry detection command:

```
python manage.py check_blood_expiry
```

Location: apps/blood/bloodinventor/management/commands/check_blood_expiry.py

Behaviour:

- Scans all BloodInventory records where status='available'
- Creates ExpiryAlert(alert_type='expiring_soon') for units expiring within 7 days
- Creates ExpiryAlert(alert_type='expiring_critical') for units expiring within 3 days
- Creates ExpiryAlert(alert_type='expired') for units already past expiry_date
- Sets expiry_alert_sent=True on BloodInventory to prevent duplicate alert creation

TIP: Schedule this command to run daily in production via Windows Task Scheduler, cron job, or celery-beat.

5.8 Django Admin (Jazzmin)

The enhanced admin panel at /admin/ uses django-jazzmin for a modern sidebar UI.

- All models are registered and editable via the admin panel
- Use to create initial admin accounts, doctors, and staff profiles
- Useful for inspecting and debugging database records during development
- Provides list views, search, filters, and inline editing for related models

6. Frontend Deep Dive

6.1 React App Structure (frontend/src/)

Directory / File	Purpose
pages/	Page-level components organised by role. Each role has its own subfolder.
pages/admin/	Admin dashboard pages: AdminDashboard, NationalInventoryDashboard, DoctorsList, HospitalManagement, ExpiryAlertsPage, BloodRequestsPage, CampBloodTrackingPage, DonationsPage, AdminSettingsPage, HospitalInventoryDetail
pages/donor/	Donor pages: DonorDashboard, DonorEligibility, DonorNotifications, DonorRegistration, publicDonorScan
pages/doctor/	DoctorDashboard (full-screen, no main layout)
pages/bloodcamp/	CampDashboard (full-screen), BloodCamps (public listing)
pages/staff/	StaffDashboard (inventory officer), LabDashboard
pages/patient/	PatientDashboard
pages/public/	LandingPage, AboutUs, Services, ContactPage, Unauthorized, NotFound
pages/auth/	Login, SignUp, ForgotPassword, ResetPassword
pages/events/	Events listing page
components/	Reusable UI components: layout wrappers, navigation, ScrollToTop util
api/RoleRoute.jsx	HOC that checks user role before rendering – redirects to /unauthorized if denied
context/AuthContext.jsx	Global auth state provider using React Context + localStorage for JWT storage
hooks/	Custom React hooks (e.g., data fetching hooks)
services/	Axios API call functions grouped by feature domain
layouts/AdminLayout.jsx	Layout wrapper for all /admin/* routes (sidebar navigation)
layouts/MainLayout.jsx	Layout wrapper for public and authenticated non-admin routes
App.jsx	Root component: defines all Routes with role guards
main.jsx	Entry point: wraps App in AuthProvider

6.2 Routing and Role Guards

App.jsx defines all routes using React Router DOM v7. Protected routes use the RoleRoute component:

```
// Protecting a route with RoleRoute:
<Route path='/donor' element={
  <RoleRoute allowedRoles={['donor', 'admin']}>
    <DonorDashboard />
  </RoleRoute>
} />
```

Route Path	Allowed Roles	Notes
/ and public pages	All users	No authentication required
/donor	donor, admin	Donor dashboard and profile
/doctor	doctor, medical_officer, admin	Doctor dashboard (no main layout)
/inventory	Inventor, inventor, admin	Staff inventory dashboard (no main layout)

/bloodcamp	bloodcamp, admin	Camp organizer dashboard (no main layout)
/admin/*	admin only	Full admin panel with AdminLayout
/staff	bloodcamp, admin	Lab dashboard within main layout
/patient	doctor, medical_officer, admin	Patient view

6.3 Authentication Context (context/AuthContext.jsx)

The AuthContext provides global authentication state:

```
// Accessing auth state in any component:
const { user, login, logout, accessToken } = useContext(AuthContext)
```

Context interface:

- user: Decoded JWT payload object containing { id, email, username, role }
- accessToken: String – current valid access token
- login(email, password): Async function – authenticates, stores tokens, decodes user
- logout(): Async function – calls /auth/logout/ to blacklist token, clears localStorage

NOTE: Access tokens are automatically refreshed before expiry by the AuthContext using the stored refresh token.

6.4 API Service Layer (Axios)

Axios is configured with a base URL and automatic Authorization header injection:

```
// src/config/ or src/api/ - Axios instance
const api = axios.create({
  baseURL: 'http://127.0.0.1:8000/api/v1/',
})

// Automatic JWT injection via request interceptor:
api.interceptors.request.use(config => {
  const token = localStorage.getItem('access_token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

All API call functions in src/services/ use this configured axios instance.

6.5 Key Frontend Libraries

Library	Version	Usage in HopeDrop
react	19.2	Core UI framework and component model
react-router-dom	7.13	SPA routing with nested routes and role guards
axios	1.13	All HTTP API calls to the Django backend
framer-motion	12.35	Page transitions and component animations
recharts	3.8	Blood stock analytics charts (BarChart, LineChart, PieChart)
react-leaflet + leaflet	5.0 + 1.9	Interactive maps for hospital and camp locations

html5-qrcode	2.3	Camera-based QR code scanning at blood camps
qrcode.react	4.2	Donor QR code display and generation
jwt-decode	4.0	Decode JWT payload in browser (role extraction)
sweetalert2	11.26	Beautiful alert, success, and confirmation dialogs
dayjs	1.11	Date formatting, manipulation, and relative time display
lucide-react + react-icons	Latest	Icon sets used throughout the UI
sass	1.99	SCSS support for complex component stylesheets
tailwindcss	4.2	Utility-first CSS for rapid layout and spacing

7. Database Design

The database uses Django ORM. Development uses SQLite (db.sqlite3 in backEnd/main/). All models are defined in /backEnd/main/apps/*/models/.

Key Entity Relationships:

Relationship	Type	Description
User – Profile	OneToOne	NIC, blood group, phone number, location, hospital association
User – DonorDetails	OneToOne	QR ID, eligibility tracking, donation statistics
User – Doctor	OneToOne	Medical credentials, license number, specialisation
User – StaffProfile	OneToOne	Employee ID, designation, shift information
Hospital – BloodBank	OneToOne	Each hospital has exactly one blood bank
BloodBank – BloodInventory	ForeignKey	A blood bank holds many blood inventory items
Hospital – BloodInventory	ForeignKey	Direct denormalised FK for fast inventory queries
BloodCamp – CampRegistration	ForeignKey	One camp can have many donor registrations
CampRegistration – CampBloodCollection	OneToOne	One registration produces one collection record
BloodCamp – CampBloodCollection	ForeignKey	Track all collections per camp
BloodInventory – ExpiryAlert	ForeignKey	One unit can trigger multiple alert records
BloodInventory – InventoryChangeRequest	ForeignKey	Change requests reference the target inventory item
Doctor – BloodRequest	ForeignKey	Doctors submit blood requests
StaffProfile – BloodRequest	ForeignKey (x2)	Staff verify and approve requests (verified_by, approved_by)

Database Performance Optimisations:

- BloodInventory: Composite index on (hospital, blood_type, status) – idx_hospital_type_status
- BloodInventory: Composite index on (expiry_date, status) – idx_expiry_status
- BloodInventory: expiry_date individually indexed (db_index=True)
- All ForeignKey fields are automatically indexed by Django ORM

IMPORTANT: For production: change DATABASE to postgresql in settings.py. Install psycopg2 and set DB credentials via environment variables.

8. Testing

8.1 Running Django Tests

All tests are in apps/*/tests.py. Run from the Django project root (backEnd/main/):

```
python manage.py test
python manage.py test --verbosity=2

# With coverage:
coverage run manage.py test
coverage report
coverage html # generates htmlcov/index.html for browser viewing
```

Existing test – LiveStockApiTests (apps/blood/bloodinventor/tests.py):

```
class LiveStockApiTests(TestCase):
    def test_live_stock_returns_expected_shape(self):
        response = self.client.get('/api/v1/blood/live-stock/')
        self.assertEqual(response.status_code, 200)
        payload = response.json()
        self.assertIn('updatedAt', payload)
        self.assertIn('stocks', payload)
        self.assertEqual(len(payload['stocks']), 8) # 8 blood types: A+,A-,B+,B-,O+,O-,AB+,AB-
        first = payload['stocks'][0]
        self.assertIn('bloodType', first)
        self.assertIn('units', first)
        self.assertIn('status', first)
```

Template for new tests:

```
from django.test import TestCase
from django.contrib.auth import get_user_model

User = get_user_model()

class MyFeatureTest(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            username='testuser',
            email='test@example.com',
            password='testpass123',
            role='donor'
        )
        self.client.login(email='test@example.com', password='testpass123')

    def test_my_endpoint_returns_200(self):
        response = self.client.get('/api/v1/donor/profile/')
        self.assertEqual(response.status_code, 200)
```

8.2 Code Quality and Linting

Tool	Run Command	Config	Purpose
flake8	flake8 .	/.flake8	PEP8 style compliance for Python
ruff	ruff check .	pyproject.toml [tool.ruff]	Fast Python linter (selects E,F,W,I rules; ignores migrations)
black	black .	pyproject.toml	Auto-format Python code to PEP8
autopep8	autopep8 --in-place .	N/A	Auto-fix PEP8 issues
ESLint	npm run lint	eslint.config.js	JavaScript and React/JSX linting
coverage	coverage run manage.py test	N/A	Measure test coverage by line

8.3 CI/CD Pipeline (GitHub Actions)

GitHub Actions workflow runs on every push and pull request:

- Step 1: Checkout repository code
- Step 2: Set up Python 3.11 environment
- Step 3: pip install -r requirements.txt
- Step 4: Run flake8 Python linting – must pass with zero errors
- Step 5: Run python manage.py test – all tests must pass
- Step 6: Set up Node.js 18 environment
- Step 7: npm install in frontend/
- Step 8: npm run lint – ESLint must pass with zero errors

IMPORTANT: All Pull Requests to dev must pass the full CI pipeline before merging. This is enforced via branch protection rules.

9. Deployment

9.1 Docker

Backend Dockerfile (backEnd/main/Dockerfile):

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python manage.py collectstatic --noinput
EXPOSE 8000
CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "4"]
```

Build and run backend container:

```
docker build -t hopedrop-backend ./backEnd/main
docker run -p 8000:8000 -e DEBUG=False -e SECRET_KEY=your-secret hopedrop-backend
```

Frontend Dockerfile (frontend/Dockerfile):

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json .
RUN npm ci
COPY . .
RUN npm run build
# Then serve dist/ with nginx or a static file server
```

9.2 Production Checklist

Item	Action Required	Priority
DEBUG Mode	Set DEBUG=False in settings.py	Critical
SECRET_KEY	Generate a new 50-character cryptographically random key	Critical
ALLOWED_HOSTS	Add your domain name and/or IP address	Critical
Database	Switch to PostgreSQL: install psycopg2, update DATABASE settings	Critical
CORS Settings	Restrict CORS_ALLOWED_ORIGINS to production frontend URL; remove CORS_ALLOW_ALL_ORIGINS=True	Critical
Email SMTP	Configure real SMTP: EMAIL_BACKEND smtp, HOST, PORT, TLS, credentials via env	High
Credentials	Move all secrets to environment variables using python-dotenv	Critical
HTTPS / SSL	Set SESSION_COOKIE_SECURE=True and CSRF_COOKIE_SECURE=True	Critical
Static Files	Run python manage.py collectstatic (WhiteNoise already configured)	High
WSGI Server	Use gunicorn (not manage.py runserver) – see Dockerfile CMD	Critical
Expiry Command	Schedule check_blood_expiry daily via cron or celery-beat	High
Logs	Configure Django logging to file or external	Medium

	service (e.g., Sentry)	
--	------------------------	--

10. Adding New Features – Developer Workflow

Follow this standard workflow for every new feature:

- Step 1: Create feature branch from dev

```
git checkout dev
git pull origin dev
git checkout -b feature/your-feature-name
```
- Step 2: Add Django model (if new data needed)

```
# Create model in apps/<app>/models/new_model.py
# Register in apps/<app>/models/__init__.py
# Register in admin.py for visibility
python manage.py makemigrations <app_label>
python manage.py migrate
```
- Step 3: Create DRF serializer in apps/<app>/serializers/
- Step 4: Create service/view in apps/<app>/services/
- Step 5: Register URL in apps/<app>/urls.py
- Step 6: Write tests in apps/<app>/tests.py and run: `python manage.py test`
- Step 7: Create React page/component in frontend/src/pages/<role>/
- Step 8: Add route in frontend/src/App.jsx with appropriate RoleRoute wrapper
- Step 9: Add API service function in frontend/src/services/
- Step 10: Manual end-to-end testing; then open a Pull Request to dev

Full example – Adding a new API endpoint:

```
# — Step 1: Model —————
# apps/donor/models/feedback.py
class DonorFeedback(models.Model):
    donor = models.ForeignKey(DonorDetails, on_delete=models.CASCADE)
    message = models.TextField()
    rating = models.PositiveSmallIntegerField(default=5)
    created_at = models.DateTimeField(auto_now_add=True)
    class Meta:
        ordering = ['-created_at']

# — Step 2: Serializer —————
# apps/donor/serializers/feedback_serializer.py
class DonorFeedbackSerializer(serializers.ModelSerializer):
    class Meta:
        model = DonorFeedback
        fields = '__all__'
        read_only_fields = ['donor', 'created_at']

# — Step 3: View —————
# apps/donor/services/feedbackService.py
from rest_framework.views import APIView
from rest_framework.permissions import IsAuthenticated
```

```

from rest_framework.response import Response

class DonorFeedbackView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        donor = DonorDetails.objects.get(user=request.user)
        qs = DonorFeedback.objects.filter(donor=donor)
        return Response(DonorFeedbackSerializer(qs, many=True).data)

    def post(self, request):
        donor = DonorDetails.objects.get(user=request.user)
        serializer = DonorFeedbackSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        serializer.save(donor=donor)
        return Response(serializer.data, status=201)

# — Step 4: URL —————
# apps/donor/urls/urls.py - add to urlpatterns:
path('feedback/', DonorFeedbackView.as_view(), name='donor-feedback'),

# — Step 5: Test —————
class DonorFeedbackTest(TestCase):
    def test_feedback_submission(self):
        # ... setup and assert ...
        pass

```


11. Troubleshooting for Developers

Problem	Likely Cause	Solution
migrate fails with OperationalError	Database locked or missing migration file	Delete db.sqlite3 in backEnd/main/ and re-run python manage.py migrate; or run makemigrations first
CORS errors in browser console	Backend not running or incorrect CORS config	Verify backend is running on port 8000; check CORS_ALLOWED_ORIGINS list in settings.py
401 Unauthorized on all API calls	JWT access token expired or not being sent in header	Check AuthContext in frontend; verify access token exists in localStorage and Authorization header is being set
403 Forbidden on a specific endpoint	User role does not match required permission	Check user.role matches allowedRoles in RoleRoute; check permission_classes in the Django view
npm install / npm run dev fails	Node version too old	Ensure Node.js 18+ is installed; run 'node --version' to verify
Vite ERR_CONNECTION_REFUSED	Django backend is not running	Start backend: cd backEnd/main && python manage.py runserver
Django admin shows no models	Model not registered in admin.py	Add 'admin.site.register(MyModel)' in the app's admin.py file
makemigrations shows no changes	Model not in INSTALLED_APPS or not in __init__.py	Add app to INSTALLED_APPS in settings.py; import model in models/__init__.py
Migration conflicts on merge	Two branches created migrations for same app simultaneously	Run 'python manage.py migrate --merge' to create a merge migration
flake8 E501 line too long	Line exceeds configured max-line-length	Check .flake8 for max-line-length setting; break long lines or add '# noqa: E501'
check_blood_expiry command not found	Management command file missing or wrong path	Verify file: apps/blood/bloodinventor/management/commands/check_blood_expiry.py exists with __init__.py files in management/ and commands/
Token not refreshing properly	Refresh token expired or blacklisted	User must log in again; check REFRESH_TOKEN_LIFETIME in SimpleJWT settings (default 1 day)
QR scan not working on mobile	Camera permission denied or device camera API unavailable	Ensure HTTPS is used (camera API requires secure context on mobile browsers)